

25th October, 2024

UNIT - 3

L1 # Public Key Cryptography (Asymmetric) (diff key)

→ Symmetric Key: same key used to encrypt / decrypt

Advantages: *

- * Extremely Secure
- * Relatively Fast

Disadvantages: *

- * exchange of key
- * no. of keys required
- * wide damage if compromised

→ Key Distribution

* use symmetric key distribution protocols.

* use trusted third party (TTP).

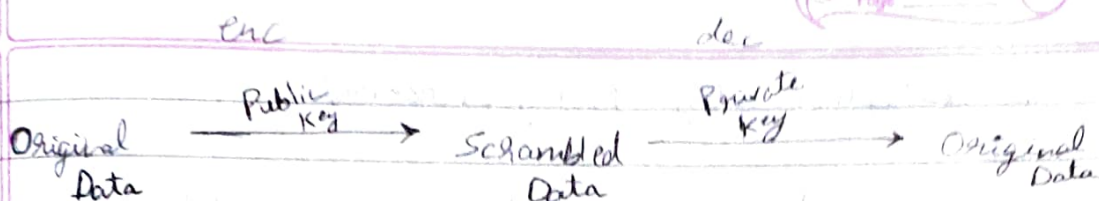
Each user has a unique secret key shared with TTP

* When 2 users would like to communicate, they establish a shared secret key = session key by interacting with TTP

Public Key Cryptography : 2 diff keys

1: enc. of plainText

1: dec. of cipher text

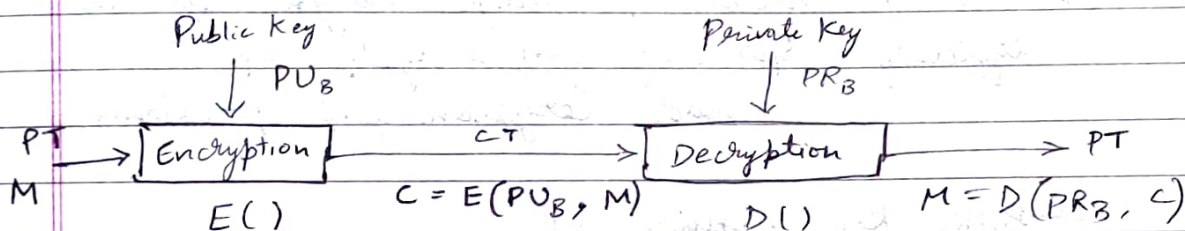


Public - Key:

- * used for encryption, verification of sign.
- * can be distributed freely
i.e. anyone can encrypt / verify.

Private - Key:

- * used for decryption, signing
- * should be kept private



user A : sender, user B : receiver
 encrypt using receiver's public key, PU_B
 decrypt using receiver's private key, PR_B
 only B has

→ Security mechanisms of PKC

- Key Distribution (eg: Diffie-Hellman key exchange)
w/o pre-shared secret (key)
- Non repudiation & Digital Signatures (eg: RSA, DSA, ECDSA)
provides msg integrity
- Identification : using challenge response protocols
with digital sign

iv) Encryption (eg. RSA, ElGamal)

v) Disadvantage: computationally very intensive

PKC consists of:

- i) Key gen algo
- ii) enc. algo
- iii) dec. algo

→ Security of Public Key

* Brute Force theoretically possible

* Keys used are too large (>512 bits)

* Security relies on large enough diff in difficulty b/w easy (en/decrypt) & hard (cryptanalyse) problems.

→ Public Key Requirements

Need for ~~trapdoor~~ one-way func.

One-way func.

$$Y = f(X) \text{ easy}$$

$$X = f^{-1}(Y) \text{ infeasible}$$

$Y = f_k(X)$: easy, if k & X are known

$X = f_k^{-1}(Y)$: easy, if k & Y are known

$X = f_k^{-1}(Y)$: infeasible, if Y known but not k

* One-way function: $f(x)$ is a func. from a set X to a set Y such that $f(x)$ is easy to compute for all $x \in X$, but for "essentially all" elements $y \in Y$ it is "computationally infeasible" to find $x \in X$ such that $f(x) = y$.

* Trapdoor one-way func: $f(x)$ is a one-way func. $f: X \rightarrow Y$ such that if one knows some specific info T , called trapdoor info, then $f(x)$ is computationally easy to invert f .

i.e. for any $y \in Y$ it is easy to find $x \in X$ such that $f(x) = y$

without trapdoor info T , $f(x)$ is one-way info

→ Types of One-way func.

1. Strong One way: easy to compute, hard to invert on all but negligible fracs. of IP
2. Weak One way: easy to compute, easy to invert on some negligible fracs of IP

→ Relationship b/w 1-way func. & crypto

* Secure enc & MAC schemes imply/require existence of 1-way funcs.

$$p=13 \quad q=17$$

$$N=221$$

$$de \bmod 221 = 1$$

$$e=35$$

$$\phi(N)=192$$

$$d(35) \bmod 221 = 1$$

One way func eg : Hash func
RSA

15th November, 2024

L2 RSA : Rivest Shamir Adleman

Encryption : (5, 14)

Text : B $\rightarrow$ 2

$$2^5 \bmod 14 = 32 \bmod 14 \\ = 4$$

ciphertext : D

Decryption : (11, 14) ^{same}
D $\rightarrow$ 4

$$4^{11} \bmod 14 = 4194304 \bmod 14 \\ \downarrow \\ \div 14 = \underline{299593.1429} \\ - 299593 \\ \hline 0.142857 \\ \times 14 \\ = 1.9999998 \\ = 2 \\ = B$$

→ How does it work?

1. Pick 2 random prime numbers

remain private
 $p=2, q=7$

2. Find product of p, q $N=14$ (publicise)

3. $\phi(N)$

cancel common factors

①

~~2~~

③

co prime with 14 : 1, 3, 5, 9, 11, 13

~~4~~

(no common factors with 14) 6 numbers

⑤

~~6~~

~~7~~

~~8~~

⑨

~~10~~

~~11~~

~~12~~

13

~~14~~

$$\begin{aligned}\text{phi func: } \phi(N) &= (p-1)(q-1) \\ &= 1 \times 6 \\ &= \underline{6}\end{aligned}$$

4. Choose e $\left\{ \begin{array}{l} 1 < e < \phi(N) \\ \text{co prime with } N, \phi(N) \end{array} \right. \rightarrow 2, 3, 4, 5$

$e=5$ LOCK : (5, 14)

5. Choose d : $de \pmod{\phi(N)} = 1$
 $5d \pmod{6} = 1$

~~5, 10, 15, 20, 25~~

5, 4, 3, 2, 1, 0

$d=11$

RSA : most widely used public key algo

Encryption given public key $(n, e) = k_{pub}$
Plaintext x

encryption func $y = e_{k_{pub}}(x) \equiv x^e \pmod n$

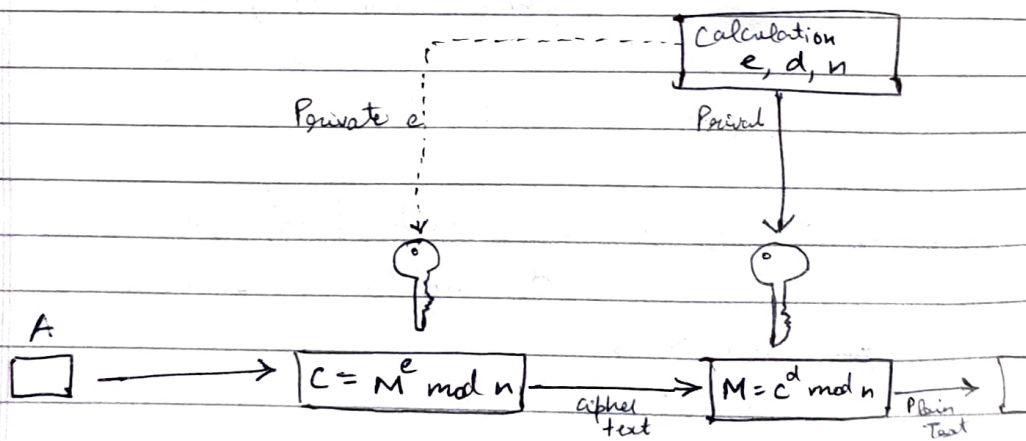
$x, y \in \mathbb{Z}$

Decryption given private key $d = k_{pr}$
cipher text y

decryption func. is $x = d_{k_{pr}}(y) = y^d \pmod n$
 $x, y \in \mathbb{Z}$

e : encryption exponent / public exponent

d : decryption exponent / private exponent



q1. How to choose large primes p, q ?

Prime No. Theorem: There are approximately

$$\frac{2^d}{\log 2^d} - \frac{2^{d-1}}{\log 2^{d-1}} \quad d \text{ bit primes}$$

q2. Relation b/w p & q

- * difference $|p - q|$ should neither be too small nor too large
- * $\gcd(p-1, q-1)$ should not be large
- * both $p-1$ & $q-1$ should contain large prime nos
- * q, p should be safe primes
 $\Rightarrow \frac{(p-1)}{2} \text{ \& \> } \frac{(q-1)}{2} \text{ are also primes}$

q3. choosing c & d

- * neither d nor c should be too small
- * d should not be smaller than $n^{1/4}$

$$5^9$$

$$\begin{array}{r} 1001 \\ (5^2)^2 (5^2)^2 \leftarrow (5^4)^2 (5) = \\ = 5^8 \times 5 = 5^9 \end{array}$$

we start with 1 & go left

if we see 0 we square

if we see 1 we square & multiply

$$x^{26} = x^{11010}$$

$$\begin{array}{r} \checkmark \checkmark \checkmark \checkmark \\ \underline{11010} \end{array}$$

$$(x^2)x = x^3 \rightarrow x^6 \rightarrow x^{13} \rightarrow x^{26}$$

Square & Multiply for modular exponentiation

Input :

base element x

exponent $H = \sum_{i=0}^t h_i 2^i$ with $h_i \in \{0, 1\}$ & $h_t = 1$
and modulus n

Output : $x^H \bmod n$

Initialization : $z = x$

Algorithm :

For $i = t-1$ down to 0

$$z = z^2 \bmod n$$

if $h_i = 1$

$$z = z \cdot x \bmod n$$

Return (z)

Complexity : #SQ = t

#MUL = $0.5t$

→ RSA security

Brute force key search
mathematical attacks

timing attacks

chosen ciphertext attacks

L3 Diffie - Hellman Key exchange

PKC solves : 1. Key Distribution
2. Digital signatures

PKC relies on: $\times$ it is computationally infeasible to determine the decryption key, given only knowledge of cryptographic algo. & enc key

Algorithm:

1. Select q (prime no.) & α (primitive root of q)

2. User A key generation: select X_A , $X_A < q$
Calculate public key Y_A

$$Y_A = \alpha^{X_A} \text{ mod } q$$

Y_A is shared with user B

3. User B key generation: select X_B , $X_B < q$
Calculate public key Y_B

$$Y_B = \alpha^{X_B} \text{ mod } q$$

Y_B is shared with user A

4. Calculate secret key of user A: $K = (Y_B)^{X_A} \text{ mod } q$

5. Calculate secret key of user B: $K = (Y_A)^{X_B} \text{ mod } q$

L4

$$3^{29} \bmod 17 \xrightarrow{\text{Easy}} 12$$

Discrete
logarithmic
Problem

$$3^? \bmod 17 \xleftarrow{\text{Hard}} 12$$

$$\mathbb{Z}_p^* = \{1, \dots, p-1\}$$

p : is an odd prime no.

Notation: $\alpha \cdot \beta = (\alpha \beta \bmod p)$

Every element in $\mathbb{Z}_p^*$ has an inverse

eg: $p = 13$

$$\mathbb{Z}_{13}^* = \{1, 2, 3, \dots, 12\}$$

$(\mathbb{Z}_p^*, \cdot)$ is a group

Special group - cyclic group

Generator

$\alpha \in \mathbb{Z}_p^*$ is a generator if

$$\begin{aligned} \{\alpha, \alpha^2, \dots, \alpha^{p-1}\} &= \mathbb{Z}_p^* \\ &= \{1, 2, \dots, p-1\} \end{aligned}$$

Cyclic group is a group with at least 1 generator

L5 Elliptic Curve Cryptography (ECC)

based on algebraic structures of elliptic curves over finite fields

difficulty of Elliptic Curve Discrete Logarithm Problem (ECDLP)

+ ECC is successor of RSA

reason: ECC uses smaller keys & signatures

very fast key generation

fast key agreement

fast signatures

→ ECC's Trapdoor

Public key: starting point A , Ending point E

Private key: no. of hops from A to E

→ ECC algorithms

1. Elliptic Curve Discrete Signing Algorithm (ECDSA)
- Digital signature

2. Elliptic Curve Diffie-Hellman (ECDH)
- key agreement

→ Applications of ECC

1. Wireless communication devices
2. Smart cards
3. web servers

→ Benefits of ECC

shorter key length: enc, dec, sign, verif. speed up storage & bandwidth savings

L6 Hashing in PKC

Message M (arbitrary length)



Hash Value

(Fixed Length)

→ Properties of Strong Hash Algorithm

1. Determinism
2. Pre-Image Resistance
3. Collision Resistance
4. Avalanche Effect
5. Hash speed

→ Types

Merkle-Damgård scheme

Message Digest

Secure Hash Algorithm

Hash func. based on block cipher

Rabin scheme

David Meyer scheme

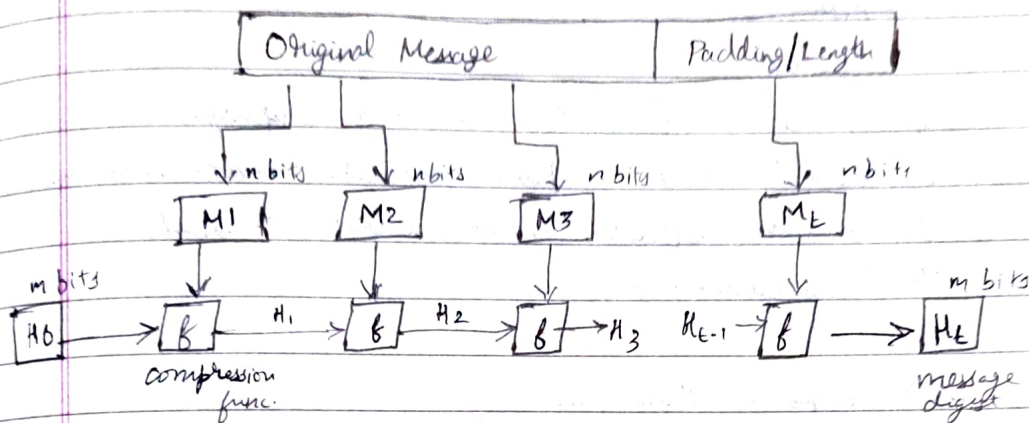
Matyas Meyer Oseas scheme

Miyaguchi-Preneel scheme

→ Merkle-Damgård Scheme (MD)

used in SHA-1, SHA-256

$$f(H_{i-1}, M_i) = H_i$$



→ Other Hash func uses

1. PRF
2. PRNG - Verman cipher / OTP
3. to create one way password file store hash of pwd & not actual pwd
eg: UNIX, cyworlds
4. for intrusion detection & virus detection - keep & check hash of files on system
eg: Tripwire

→ Lamport OTP

* Password Safety in distributed system

- Server compromise doesn't compromise password
- interception of authentication exchange doesn't compromise password either

→ Hash func Requirements

1. Variable IP size
2. Fixed OP size
3. Efficiency
4. Pre-Image Resistant (one-way property)
5. Second Pre image Resistant (weak collision resistant)
6. Collision resistant (strong collision resistant)
7. Pseudorandomness

→ Attacks on Hash func.

1. brute force attacks & cryptanalysis $\frac{n}{2}$
2. preimage / second pre image attack
3. collision resistance

MD5

Message Digest

↳ 128 bit hash value

has vulnerabilities

can be used as a checksum to verify data integrity

basic req: should be computationally infeasible
to find 2 distinct messages that hash
to same value.

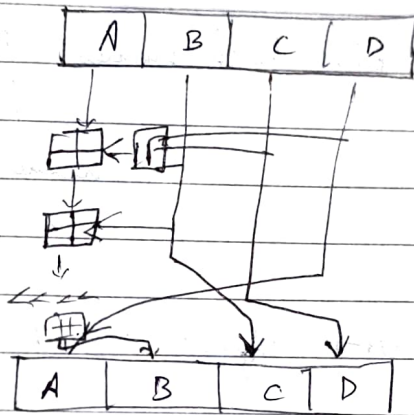
MD5 fails

→ Merkle-Damgård construction

* fixed OP 128 bits
variable len IP

* message is broken down into chunks of 512 blocks
(16 32bit words)

* message is padded so that it is divisible by 512



→ Implementation

1. Append Padding bits
2. Append Length
3. Initialize MD buffer
4. Process message in 16 word - blocks

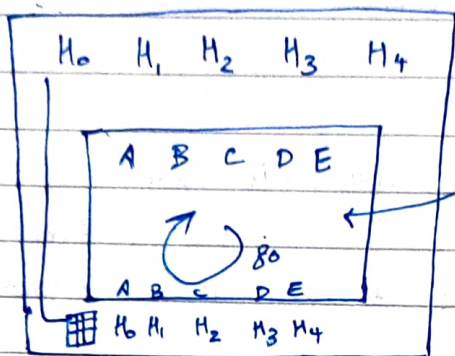
→ MD5 Flaws

1. Rainbow tables for passwords
2. COLLISION

16th November, 2024

SHA-1

$M = 512$ bits

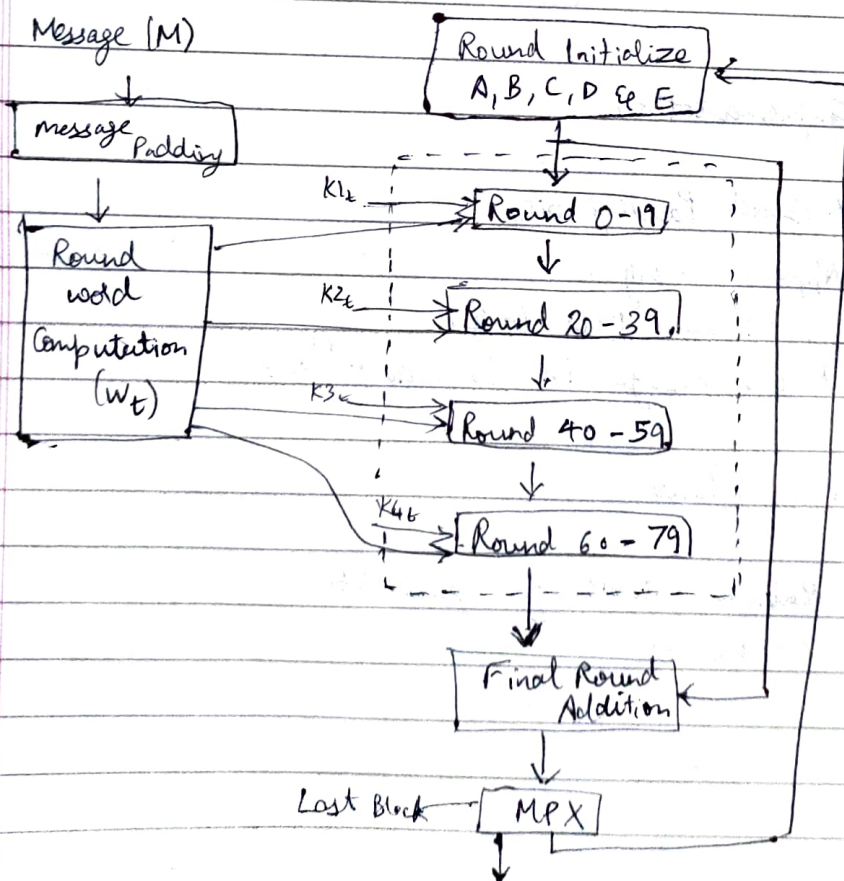


* produces 160 bit hash values (message digest)

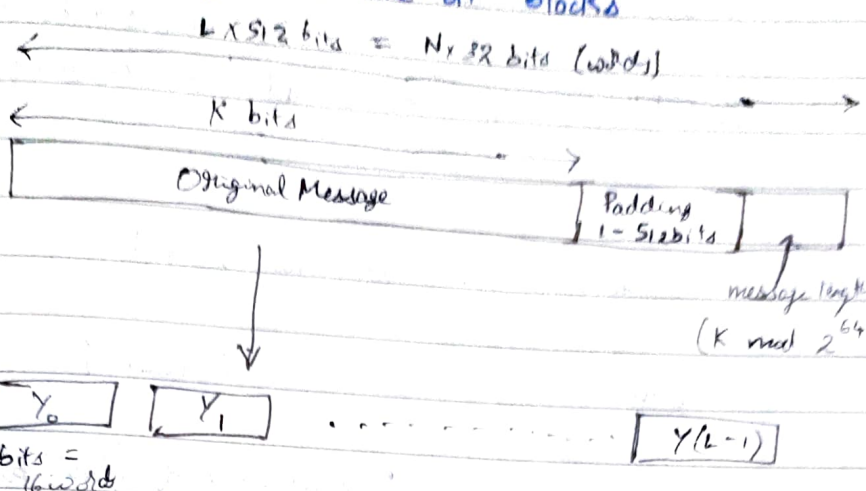


hexadecimal no.
(40 digits)

SHA-1 Algorithm Block Diagram



→ Divide the message into 512-bit blocks



→ 1 iteration of SHA → Process Blocks

~~abcde = (e +~~

1. Copy chaining vars A - E into vars a - e

2. Divide current 512 bit block into 16 blocks of 32 bits

3. SHA has 4 rounds, 20 steps

4. Each round has 3 ops: 512 bit block
The register abcde
constant $K[t]$ ($t = 0 \dots 79$)

5. SHA has total 80 iterations ($4 \times 20 \text{ iters}$)
rounds

→ 1 iteration of SHA

$abcde = (e + \text{Process } P + S5(a) + W[t] + K[L]),$
 $a, S30(b), c, d$

abcde : register of 5 vops

Process P : logic operations

St : Circular Left Shift of 32 bit subblock by t bits

$W[t]$: A 32 bit derived from the current
32 bit sub block

$K[t]$: One of the 5 additive constants

→ Process P in each SHA round

Round

Process P

1

$(b \text{ AND } c) \text{ OR } (\text{NOT } b \text{ AND } d)$

2

$b \text{ XOR } c \text{ XOR } d$

3

$(b \text{ AND } c) \text{ OR } (b \text{ AND } d) \text{ OR } (c \text{ AND } d)$

4

$b \text{ XOR } c \text{ XOR } d$

→ Working Procedure

1. Message (M)
2. Message Padding
3. Round Wdd Computation ($W_t, W_{t+1}, \dots, W_t$)
4. Round Initialize (A, B, C, D & E)
5. Round constants ($K_t, K_{t+1}, \dots, K_t$)
6. Rounds (0-79)
7. Final Round Addition
8. MPX (Multiplexing)

Summary SHA-1

Digest Sizes: 160 bits

Block Sizes: 512 bits

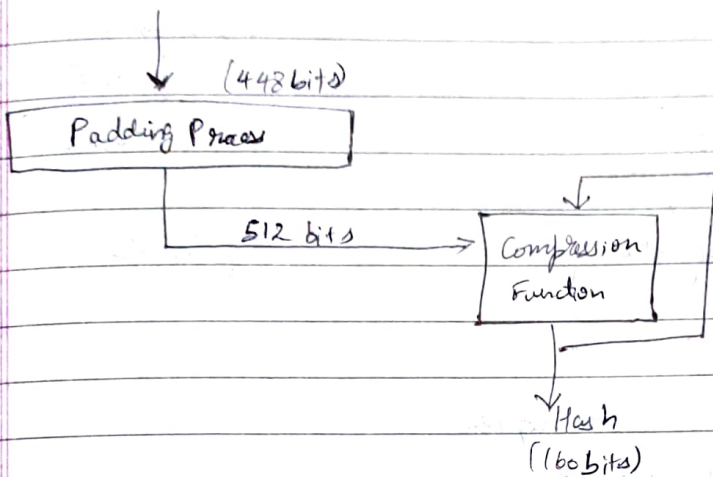
Certification: FIPS PUB 180-4, CRYPTREC (non-ideal)

Rounds: 80

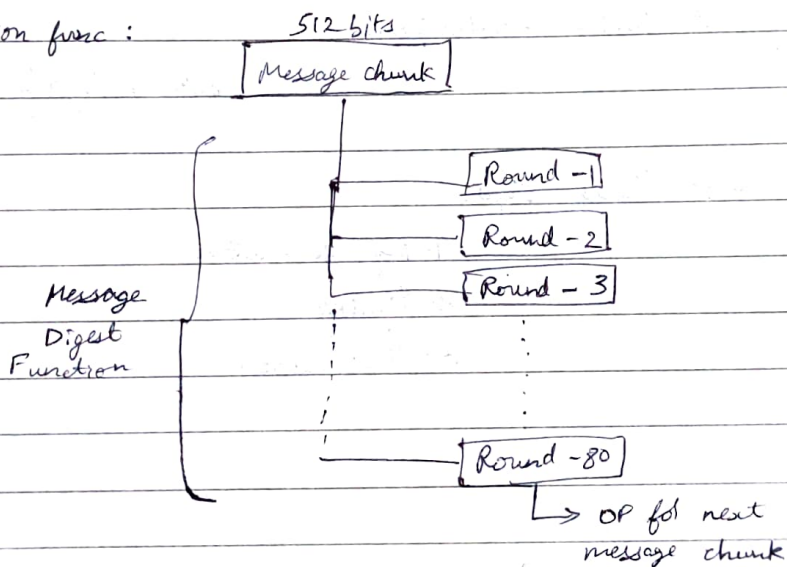
Structure: Merkle - Damgård construction

Arbitrary Message $\rightarrow$ SHA-1 Hash $\rightarrow$ Hash (160 bits)

$X (x_1, x_2, x_3, \dots, x_n)$



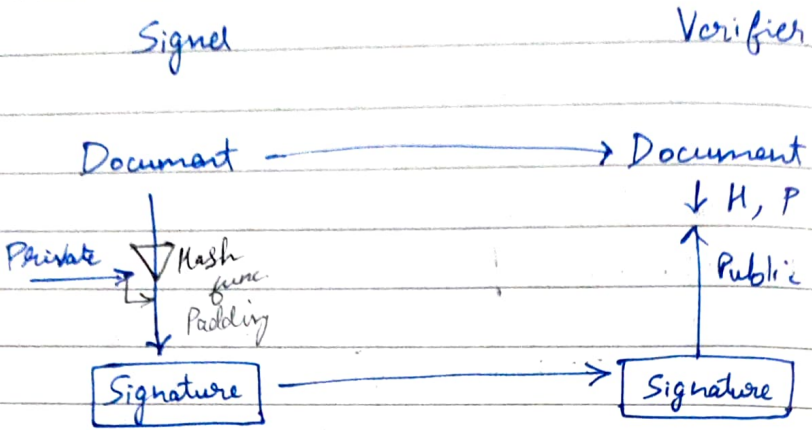
Compression func :



Variations & Parameters	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message digest	160	224	256	384	512
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block size	512	512	512	1024	1024
Word size	32	32	32	64	64
no. of steps	80	64	64	80	80

L8

Digital Signature

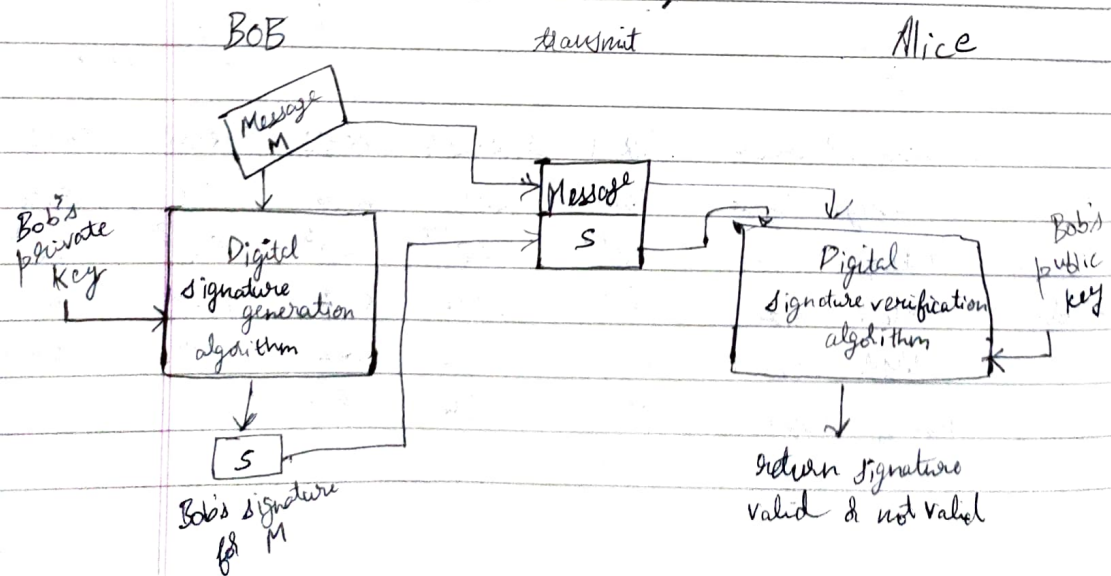


- * Easy to make copies
- * Fast distribution
- * copies are as good as original
- * Easily modifiable

Lack integrity & authenticity

→ Process steps

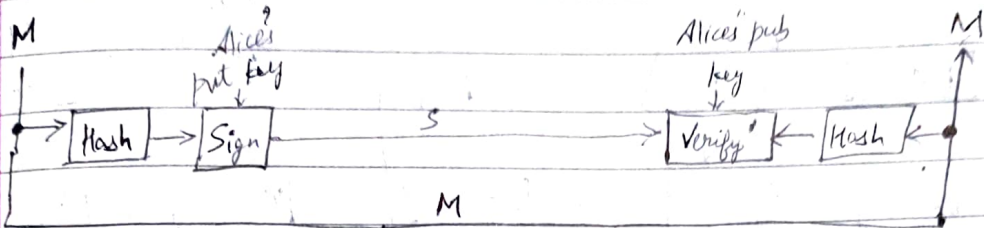
1. Key Generation
2. Signing
3. Verification



→ Signing the Digest

Alice

Bob



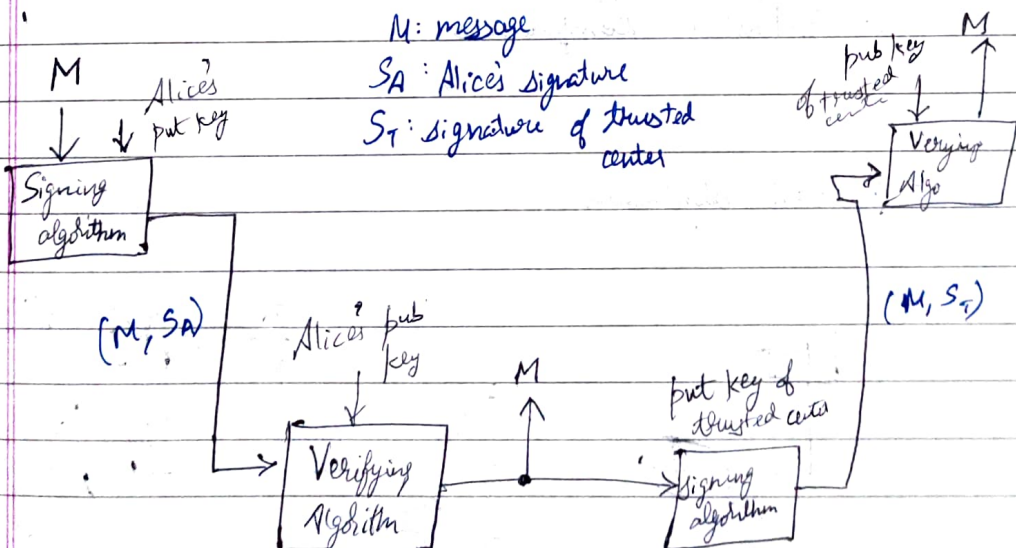
→ Services

1. Message Authentication
2. Message Integrity
3. Non repudiation
4. Confidentiality

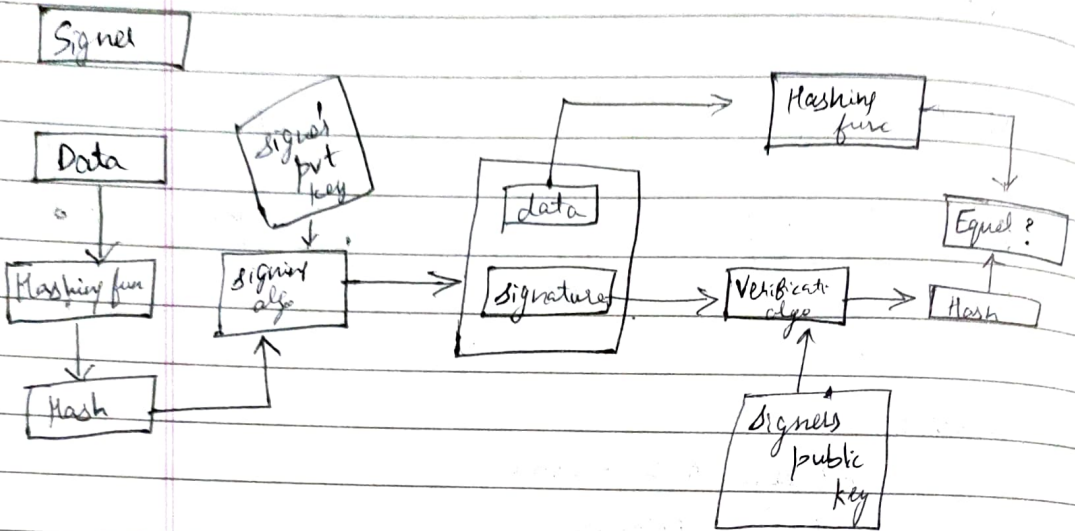
→ Using a trusted center for non repudiation

Alice

Bob



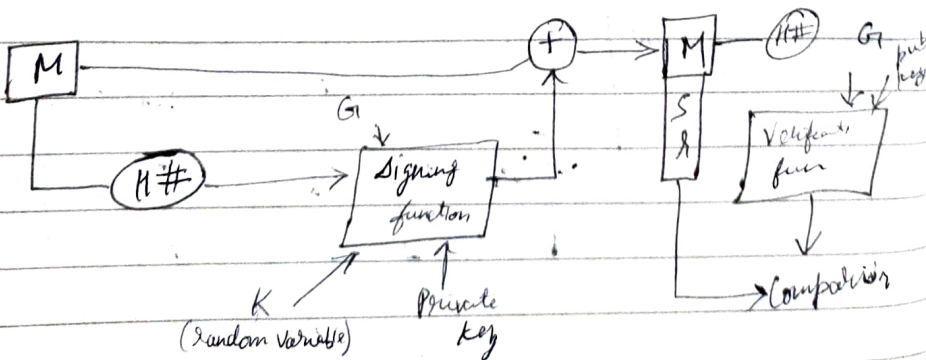
→ Block Diagram



→ Importance of Digital Signatures

1. Ensures Authenticity
2. Offers non-repudiation
3. Provides Security
4. Improves efficiency
5. Enhances Compliance

→ DSA algorithm



→ Cons of using Digital Signature Algorithm

1. Limited Key Length
2. Rigid Key Management
3. Limited Digital certificate support
4. Incremental signature algorithm
5. Relatively new
6. Impact on storage & transmission

→ Advantages of DSA

1. Highly robust
2. Better speed
3. Less storage
4. Patent Free

RSA Signatures

digital signature scheme (Sign + Verify)

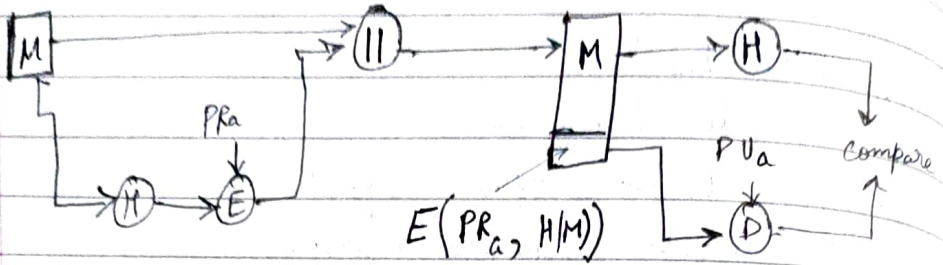
math of modular exponentiations & discrete log
& difficulty of integer factorization problem (IFP).

RSA sign/verify process

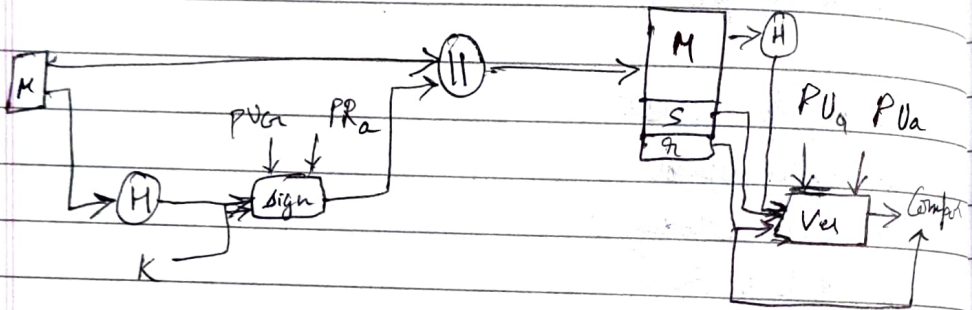
RSA sign: Message Hash
↓
encrypts with priv key
↓
signature (integer)

RSA verify: Message Hash
↓
decrypts with pub key
↓
compares Hashes
↓
signature valid & not

RSA approach



DSA approach



Digital Certification

- TLS/SSL
- Client certificates
- Code signing certificates

→ Challenges & Considerations

- Certificate management
- Private key security
- CA trustworthiness
- Revocation effectiveness